

Partie 1 :

Critere	Langage compile (C, C++, Rust)	Langage interprete (Python, JS)
Quand la traduction se fait-elle ?	À la compilation	A chaque exécution
Vitesse d'execution	Rapide car déjà compilé	Plus lente car interprété au fur et à mesure
Portabilite du binaire	Depend de l'OS	Fonctionne partout avec un interpreteur
Detection des erreurs	Avant exécution	Pendant exécution
Besoin d'un runtime installe ?	Non, .exe autonome	Oui, Python, NodeJS, navigateur etc
Exemples de langages	Java	GoLang

1. Pourquoi un programme C est-il generalement plus rapide qu'un programme Python equivalent ?

Car le programme en C est déjà compilé donc la machine n'a pas à interpréter le code pendant l'exécution, de même pour les allocations mémoire etc.

2. Que se passe-t-il si vous compilez un programme C sur Windows et essayez d'executer le fichier resultant sur Linux ?

Très probable que ça ne fonctionne pas car le .exe sera mal lu par linux, les bibliothèques sont différentes et de même pour le format binaire.

3. Les interpreteurs Python, Ruby et JavaScript sont eux-memes ecrits en C/C++. Que cela nous dit-il sur l'importance des langages compiles ?

ils sont très efficace et permettent une exécution fiable et rapide.

4. Donnez deux exemples concrets d'applications ou la performance impose l'usage d'un langage compile (pensez aux domaines vus en cours : systemes, embarque, jeux...).

- Système embarqué d'un avion de chasse, type F-16 ou Rafale. Système complètement autonome et sans erreur pendant l'exécution.
- système d'un sous-marin, tout aussi autonome.

Partie 2 :

2.1 Verifier l'installation

```
jesui@ANV15C:~/langage_compiles$ gcc --version
gcc (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0
Copyright (C) 2023 Free Software Foundation, Inc.
This is free software; see the source for copying conditions. There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.
```

Je suis sur Ubuntu 24.04 en WSL sur VScode depuis Windows 11.

Partie 3 -- Votre premier programme C :

3.1 Anatomie d'un programme C

3.2 Compiler et executer

3.3 Le code de sortie

3.4 Exercices progressifs

Exercice A -- Hello World

```
● jesui@ANV15C:~/langage_compiles/jour1$ echo $?
0
```

Exercice B -- Affichage personnalisé

```
● jesui@ANV15C:~/langage_compiles/jour1$ ./presentation
=== Mon premier programme C ===
Prenom : Cyril
Niveau : Bachelor 1
Ecole : YNOV Montpellier
```

Exercice C -- Variables et affichage

Exercice D -- Code de sortie personnalisée

En convention Unix, un **code de sortie différent de 0** signifie qu'il y a eu une **erreur** ou un **échec** lors de l'exécution du programme.

Partie 4 -- Identifier et corriger des erreurs

4.3 Debogage : corrigez ces programmes

Programme 1 -- Erreur de syntaxe :

```
#include <stdio.h>

int main(void) {
    printf("Bonjour le monde!")
    return 0;
}
```

Oubli du “;” à la fin de la ligne 4

Programme 2 -- Mauvais formats :

```
#include <stdio.h>

int main(void) {
    int nombre = 42
    float pi = 3.14;
    printf("Nombre : %f, Pi : %d\n", nombre, pi);
    return 0;
}
```

%f et %d sont inversés, et il manque un “;” à la fin de la ligne 4

Programme 3 -- Bug a l'execution :

```
#include <stdio.h>

int main(void) {
    int a = 10;
    int b = 0;
    int resultat = a / b;
    printf("Resultat : %d\n", resultat);
    return 0;
}
```

On ne peut pas diviser par 0, b est égal à zero et la compilation le détecte.

Programme 4 -- Bug logique :

```
#include <stdio.h>

int main(void) {
    int score = 85;
    if ((score) = 100) {
        printf("Score parfait !\n");
    }
    return 0;
}
```

Oubli de parenthèses à la ligne 4

4.4 Compiler avec les warnings

Pourquoi est-il recommandé de toujours compiler avec -Wall -Wextra ? :

Ces options activent tous les avertissements du compilateur. Ils détectent les erreurs logiques non bloquantes : variables inutilisées, formats incorrects, comparaisons suspectes, etc.

Partie 5 -- Mini-projet : ma fiche d'identite

```
C fiche.c
1  #include <stdio.h>
2
3  int main(void) {
4      char prenom[] = "Cyril";
5      int age = 19;
6      float taille = 1.75f;
7      char initial = 'C';
8
9      printf("=== FICHE D'IDENTITE ===\nPrenom : %s\nAge : %d ans\nTaille : %.2f m\nInitial : %c\n", prenom, age, taille, initial);
10
11     return 0;
12 }
```